

```
using Pkg
Pkg.add("Plots")
Pkg.add("CSV")
Pkg.add("DataFrames")
Pkg.add("Dates")
```

```

Resolving package versions...
No Changes to `~/.julia/environments/v1.11/Project.toml`
No Changes to `~/.julia/environments/v1.11/Manifest.toml`
Resolving package versions...
No Changes to `~/.julia/environments/v1.11/Project.toml`
No Changes to `~/.julia/environments/v1.11/Manifest.toml`
Resolving package versions...
No Changes to `~/.julia/environments/v1.11/Project.toml`
No Changes to `~/.julia/environments/v1.11/Manifest.toml`
Resolving package versions...
Updating `~/.julia/environments/v1.11/Project.toml`
[ade2ca70] + Dates v1.11.0
No Changes to `~/.julia/environments/v1.11/Manifest.toml`

```

E1:

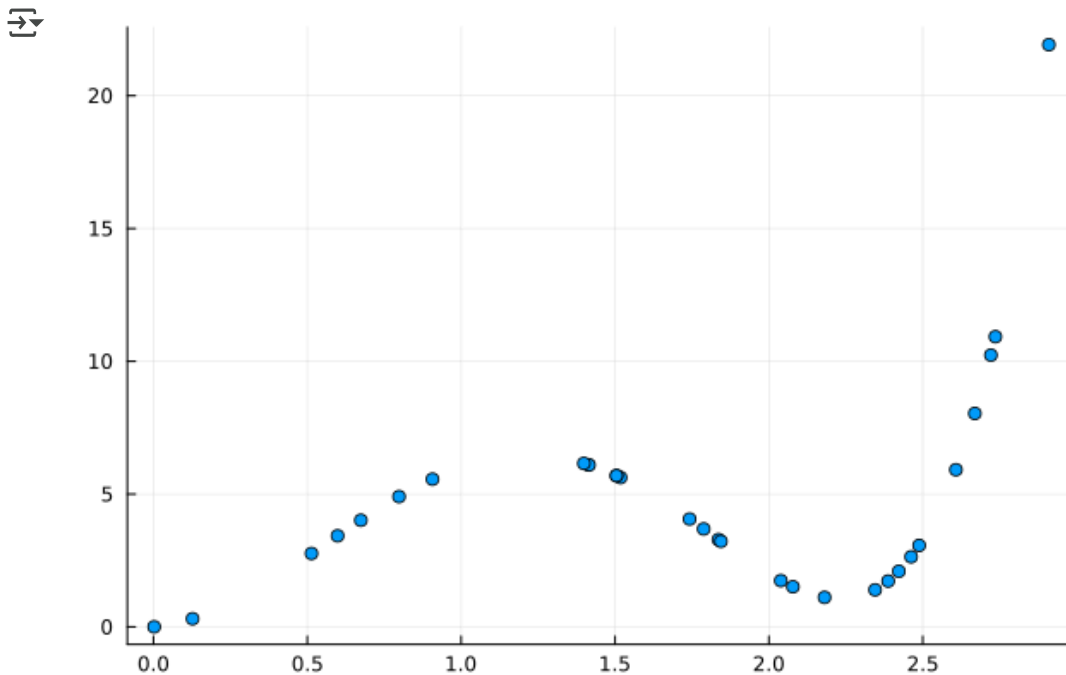
#a) polinômio: $x^5 - 2x^4 - 6x^3 + 12x^2 + x$

```
using Plots
```

```
v_y = Float64[]
v_x = Float64[]
```

```
f(x) = x^5 - 2x^4 - 6x^3 + 12x^2 + x
v_x = rand(30) * 3
v_y = f.(v_x)
```

```
scatter(v_x, v_y, leg=false)
```



#b)

```
using Plots
using Statistics
```

```
x = collect(0.0:0.1:3.0)
y = (x.^5 - 3x.^4 - 6x.^3 + 12x.^2 + x) + randn(length(x)).*10 .- 5
```

```
#vetor de erros
v_erros = Float64[]
```

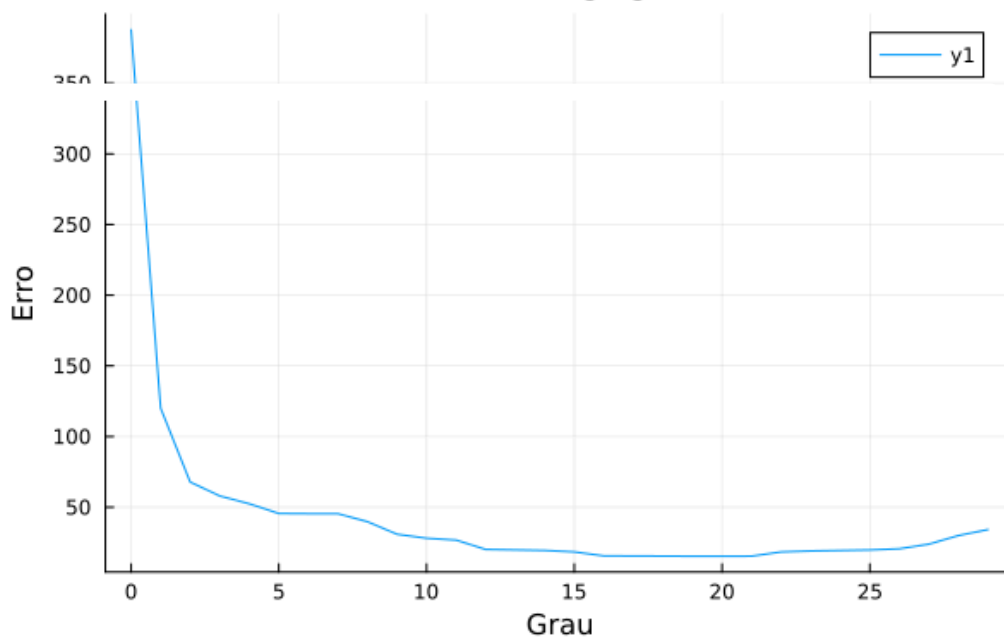
```
for grau in 0:29
    X = hcat([x.^ i for i in 0:grau]...)
    B = X \ y
    y_guess = X * B
    erro = mean((y .- y_guess) .^ 2)
    push!(v_erros, erro)
end
```

#d) o erro vai diminuindo conforme o grau aumenta, mas depois ele sobe novamente, não é o de ser esperado num mundo perfeito ele só diminuiria, mas num computador, onde aproximações são constantes, chega um ponto que os benefícios de ter um grau maior são superados pelo quanto os erros das aproximações são exarcebados. As fórmulas são instáveis.

```
plot(0:29, v_erros,
     xlabel="Grau",
     ylabel="Erro",
     title="Erro em Escala Log (grau vs erro)")
```



Erro em Escala Log (grau vs erro)



c) não, pois ao chegar no grau 30, ficamos com uma função com grau igual à quantidade de variáveis, o que acaba gerando variáveis livres, desestabilizando as matrizes utilizadas.

E2:

```
filmes = ["Toy story", "Rocky", "De volta pro futuro", "Curtindo a vida adoidado", "Os incríveis",
          "Duna", "Batman begins", "Harry Potter 1", "Shrek"]
```

```
# Toy Story = t
# Rocky = r
# De volta pro futuro = v
# Curtindo a vida adoidado = c
# Os Incríveis = o
# Duna = d
```

```
# Batman begins = b
# Harry Potter 1 = h
# Shrek = s
```

```
#matriz das comparações:
```

```
#      t      r      v      c      o      d      b      h      s
A = [
    1  -1   0   0   0   0   0   0   0;
    0   0   1  -1   0   0   0   0   0;
    0   0   0   0   1  -1   0   0   0;
    0   0   0   0   0   0   1  -1   0;
    0   0   0   0   0  -1   0   0   1;
    0  -1   0   0   0   0   0   1   0;
    1   0  -1   0   0   0   0   0   0;
    0   0   0   0   1   0   0  -1   0;
    0   0   0   1   0  -1   0   0   0;
    0   0   1   0   0  -1   0   0   0;
    0  -1   0   0   0   0   0   0   1;
    0   0   0   0   1   0  -1   0   0;
    1   0   0   0   0   0  -1   0   0;
    0   0   0  -1   1   0   0   0   0
]
```

```
#vetor das diferenças:
```

```
b = [11,
     3,
     7,
     2,
     9,
     7,
     5,
     5,
     2,
     2,
     11,
     5,
     3,
     7]
```

```
r = A \ b
```

```
ranking = sortperm(r; rev=true)
println("Ranking dos filmes:")
for i in ranking
    println("$(filmes[i]): $(round(r[i], digits=3))")
end
```

```
➦ Ranking dos filmes:
Shrek: 4.826
Os incríveis: 4.534
Toy story: 4.044
Batman begins: 0.671
Harry Potter 1: -0.564
De volta pro futuro: -0.642
Curtindo a vida adoidado: -2.519
Duna: -3.45
Rocky: -6.898
```

E3:

```
using Plots
using CSV
using DataFrames
```

```
using Dates

v_dia = Date[]
v_peso = Float64[]

cont = 0 #contador de anos
ant = 0 #mês anterior

#passando os dados prum dataframe
df = CSV.read("Pesagem - Sheet1.csv", DataFrame);

#guarda o primeiro dia
dia_mes = df[1, :Data]
dia, mes = parse.(Int, split(dia_mes, "/"))

data_base = Date(2024, mes, dia)

#separando as datas dos pesos
for row in eachrow(df)
    dia_mes = row.Data
    peso = row."Peso (kg)"

    #separa o dia do mes
    dia, mes = split(dia_mes, "/")

    #transforma eles em ints
    dia = parse(Int, dia)
    mes = parse(Int, mes)

    #aumenta o contador de anos se passar o mes
    if mes < ant
        cont+=1
    end
    ant = mes

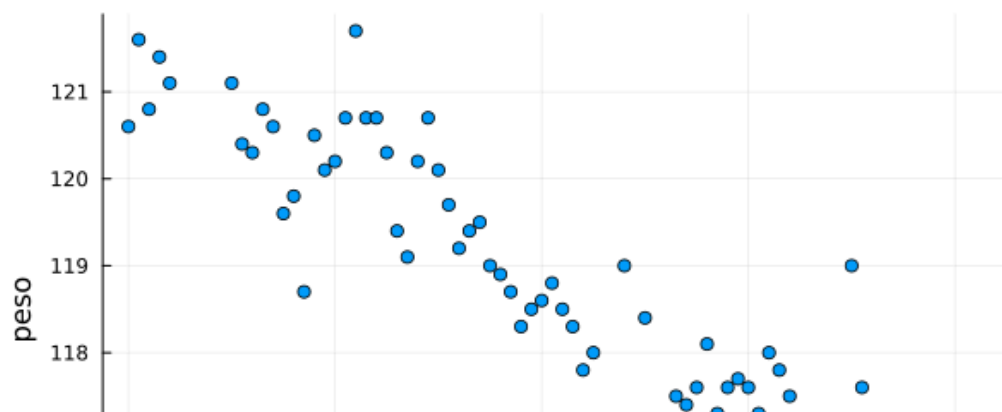
    #criando a data
    data_em_dia = Date(2024+cont, mes, dia)

    push!(v_peso, peso)
    push!(v_dia, data_em_dia)
end

#colocando todas as datas em relação à primeira
v_dia = Dates.value.(v_dia .- v_dia[1])

#criando o grafico do peso
scatter(v_dia, v_peso,
        xlabel="dia",
        ylabel="peso",
        leg=false)

#baseado nesse grafico, dá pra ver que ela tem o formato de uma
#função de primeiro grau
```



#agora para descobrir a função:

#medias

```
x = mean(v_dia)
```

```
y = mean(v_peso)
```

#agora para achar os coeficientes

```
a = sum((v_dia .- x) .* (v_peso .- y)) / sum((v_dia .- x).^2)
```

```
b = y - a * x
```

```
println("Equação: y = $a x + $b")
```

#logo, descobrimos que $y = -0.06x + 121.45$



Equação: $y = -0.06647150089703367 x + 121.4508816602138$

#agora para descobrir que dia é esse, isolamos o x:

```
dia_110 = (110 - b) / a
```

```
dia_110 = data_base + Day(round(Int, dia_110))
```

```
println("ele terá 110 kg no dia ", dia_110)
```

#isso da 16 de abril de 2025



ele terá 110 kg no dia 2025-04-16